

Orbit — Personal Career Operating System

Enterprise Software Documentation

Document type: Software Design Document (SDD) / Software Requirements Specification (SRS) / Technical Reference **Prepared for:** Final-year project submission, GitHub repository, and portfolio use **Status:** Pre-implementation — Architecture & UI design stage

A note on how this document was produced

The original request asked for documentation "derived entirely from source code." I want to be transparent about what was actually available, because that materially affects which sections below are **authoritative** and which are **inferred/recommended**:

Artifact supplied	What it actually contains
<code>orbit_full_ui.html</code>	A static HTML/CSS Flutter UI mockup — 24 screens, a style guide, no logic, no API calls, no state management code
Architecture diagram (image)	A hand-drawn system diagram showing Controllers/Routes/Services, Postgres, Redis, scraper workers, and external integrations
"Functional Dependencies" note (image)	A 10-item feature checklist
Free-text description	Problem statement + tech stack list (Flutter, Node.js, Express, Supabase, Playwright, Infisical)

No backend source code, database schema, API route files, or Flutter widget/state-management code was provided. Sections that would normally be "derived from the codebase" (endpoint specs, controller logic, DB constraints, auth middleware, exception handling, etc.) **do not exist yet** — the project is currently at the UI-mockup + architecture-sketch stage, not the implementation stage.

Per your own instruction ("if any information cannot be determined, explicitly state that it could not be inferred instead of making assumptions"), every section below is labeled:

- **Derived** — taken directly from the UI file, the diagram, or your description
- **Inferred** — a reasonable reconstruction based on the architecture diagram and stated tech stack, but not confirmed by actual code

-  **Not available** — cannot be determined at all; a recommended structure is given instead, to be replaced once code exists

This keeps the document useful today and gives you a ready-made skeleton to fill in as you write the actual backend and app.

Table of Contents

1. [Project Overview](#)
 2. [Business Requirements](#)
 3. [System Architecture](#)
 4. [Technology Stack](#)
 5. [Project Structure](#)
 6. [Database Design](#)
 7. [Backend Documentation](#)
 8. [Authentication & Authorization](#)
 9. [Frontend Documentation](#)
 10. [API Documentation](#)
 11. [Core Features](#)
 12. [Algorithms & Workflows](#)
 13. [UI/UX Documentation](#)
 14. [Security Analysis](#)
 15. [Performance Optimization](#)
 16. [Error Handling](#)
 17. [Deployment Guide](#)
 18. [Testing](#)
 19. [Software Engineering Practices](#)
 20. [Future Enhancements](#)
 21. [Challenges Faced](#)
 22. [Conclusion](#)
-

1. Project Overview

 **Derived**

1.1 Project Name

Orbit — tagline: *"Every job board. One career system."*

1.2 Project Description

Orbit is a personal, self-hosted job-search operating system that aggregates listings from multiple external job boards (LinkedIn, Indeed, Naukri, Internshala) via automated scraping and official APIs, ranks them against the user's actual skill profile, and gives the user a single dashboard to discover, track, and improve their odds of getting hired — rather than manually checking five or six separate portals every day.

1.3 Problem Statement

Job seekers currently must:

- Visit 5-6 different portals daily to find relevant openings
- Manually cross-reference postings against their own skills
- Track applications in spreadsheets or not at all
- Miss deadlines because nothing proactively reminds them
- Have no objective signal of whether their GitHub activity, DSA practice, or resume are actually competitive

This is repetitive, time-consuming, and provides no feedback loop on whether the user's effort is actually improving their hireability.

1.4 Existing System

Manual, portal-by-portal job hunting using native search/filter tools on each site (LinkedIn, Indeed, Naukri, Internshala), spreadsheets for tracking applications, and no automated linkage between coding practice (GitHub/LeetCode) and job readiness.

1.5 Proposed System

A single Flutter application backed by a Node.js/Express service that:

- Continuously scrapes/aggregates opportunities from multiple sources into one ranked feed
- Automates (where permitted) parts of the registration/application flow
- Tracks the full application lifecycle (Saved → Applied → Interview → Offer)
- Computes a composite "Career Health Score" from GitHub, LeetCode, resume performance, and application activity
- Sends deadline and streak notifications proactively via a background worker/queue system

1.6 Objectives

- Reduce time-to-apply by consolidating job discovery into one feed
- Increase interview conversion by surfacing skill gaps and ATS mismatches before submission
- Build a consistency habit loop (commits, DSA, applications) via streaks and a daily AI coach
- Automate the mechanical parts of job hunting (fetching, deduplicating, reminding, and where feasible, registering)

1.7 Scope

In scope: job aggregation from configured sources, personal dashboard, application tracking, resume/ATS tooling, GitHub/LeetCode activity tracking, salary benchmarking, notifications, single-user personalization. **Out of scope (current design):** multi-tenant SaaS features, employer-facing tooling, payment processing, applicant messaging with recruiters beyond a CRM-style note (no in-app chat is shown in the UI).

1.8 Target Users

Primarily the project owner as a personal tool; architecturally extensible to any individual job seeker, particularly students and early-career software engineers actively interviewing (the UI's persona — "Arjun Mehta, Flutter Developer" — reflects this).

1.9 Key Benefits

- One feed instead of six tabs
- Objective, numeric feedback on career readiness (Career Health Score)
- Deadline and streak automation reduces missed opportunities
- Resume performance is measured, not guessed

1.10 Features Summary

#	Feature	Source
1	Aggregated job feed via Unstop API + scraping (LinkedIn, Indeed, Naukri, Internshala)	Functional dependencies list, architecture diagram
2	Category/preference-based filtering	Functional dependencies list
3	Application history tracking (Kanban pipeline)	UI: Application kanban screen
4	Wishlist / saved jobs with expiry detection	UI: Saved jobs screen

#	Feature	Source
5	Deadline notifications	Functional dependencies list, UI: Notifications screen
6	High-demand job / skill visualization	UI: Skill gap analyzer
7	Job eligibility verification	Functional dependencies list
8	Roadmap creation & notes/PDF export	Functional dependencies list, UI: Learning roadmap
9	GitHub & LeetCode tracking in profile	UI: GitHub tracker, LeetCode tracker, Profile
10	Automated job registration	Functional dependencies list, architecture diagram (Playwright scraper worker)

2. Business Requirements

2.1 Functional Requirements 🟡 Inferred (from UI + feature list)

- FR1: The system shall aggregate job/internship postings from Unstop API and scraped sources (LinkedIn, Indeed, Naukri, Internshala) on a scheduled basis.
- FR2: The system shall compute and display a "match score" (%) per job against the user's stored skill profile.
- FR3: The user shall be able to save, bookmark, and remove jobs from a wishlist.
- FR4: The system shall track applications through defined pipeline stages: Saved → Applied → Interview → Offer.
- FR5: The system shall detect and flag expired/removed listings for saved jobs.
- FR6: The system shall notify the user of upcoming application deadlines and interview rounds.
- FR7: The system shall connect to and pull activity data from GitHub and LeetCode accounts.
- FR8: The system shall compute a composite Career Health Score from GitHub, LeetCode, Projects, Applications, and Resume dimensions.
- FR9: The system shall scan an uploaded resume against a target job description and report an ATS compatibility score with missing/present keywords.
- FR10: The system shall compare the user's declared skills against aggregated demand data across saved/applied jobs and surface high-priority gaps.

- FR11: The system shall provide salary benchmarking by role, city, and year-over-year trend.
- FR12: The system shall generate a daily, personalized set of recommended actions ("AI Coach").
- FR13: The system shall support multiple resume versions and track interview-conversion rate per version.
- FR14: The system shall support user-configurable job preferences (locations, salary range, job type) and notification toggles.
- FR15: Where technically and legally feasible, the system shall automate submission of job applications via scripted browser automation.

2.2 Non-Functional Requirements ● Inferred

Category	Requirement
Performance	Feed refresh should complete scraping cycles without blocking the user-facing API (handled via background workers/queue per architecture diagram)
Scalability	Worker/queue separation (Redis-backed) should allow scraper load to scale independently of the API
Reliability	Scraping failures (source site layout changes, rate limits) must not crash the aggregation pipeline
Usability	Mobile-first, single-hand-reachable navigation (bottom nav, 4 primary tabs)
Maintainability	Layered backend (Controllers → Routes → Services) per architecture diagram to isolate scraping/business logic from HTTP handling
Security	Secrets (API keys, credentials) managed via Infisical rather than hardcoded/committed
Portability	Cross-platform mobile client via Flutter (iOS/Android from one codebase)

2.3 User Roles ● Derived

Orbit is currently a **single-user personal system** — there is no evidence in the UI or architecture of multi-role support (no admin panel, no recruiter role, no team/org concept). One role: **Job Seeker (owner/user)**.

2.4 User Stories ● Inferred from UI

- As a job seeker, I want to see all new matching jobs from every board in one feed, so I don't have to check each site separately.

- As a job seeker, I want to filter by remote/full-time/salary/skill so irrelevant postings don't clutter my feed.
- As a job seeker, I want a kanban view of my applications so I always know what stage each one is in.
- As a job seeker, I want to be warned before a saved job expires so I don't waste time applying to a closed role.
- As a job seeker, I want to know which of my resumes actually gets interviews, so I know which one to send.
- As a job seeker, I want my resume checked against a specific job description so I can fix keyword gaps before applying.
- As a job seeker, I want a single score that tells me if my GitHub, DSA, and resume are strong enough, so I know where to focus.
- As a job seeker, I want a daily short list of actions (solve X, commit today, apply to Y more) so I stay consistent.
- As a job seeker, I want to see realistic salary ranges by city for my role, so I can negotiate confidently.

2.5 Use Cases Inferred

Use Case	Actor	Trigger	Outcome
Browse aggregated feed	User	Opens app	Ranked job list with match scores
Save a job	User	Taps bookmark	Job added to Saved/Wishlist, moves to "Saved" kanban column
Track application	User	Marks job "Applied"	Job moves to "Applied" column; history recorded
Scan resume vs JD	User	Uploads resume + selects job	ATS score + keyword gap report generated
View skill gap	User	Opens Skill Gap screen	Aggregated demand across saved/applied jobs vs profile skills
Receive deadline alert	System (scheduled)	Cron/notification worker fires	Push notification delivered

Use Case	Actor	Trigger	Outcome
Connect GitHub/LeetCode	User	Taps "Connect" in Profile	Background sync worker begins pulling activity
Auto-register for a job	System (Playwright worker)	User opts in / triggers automation	Application submitted on external portal

2.6 Acceptance Criteria 🟡 Inferred (representative example)

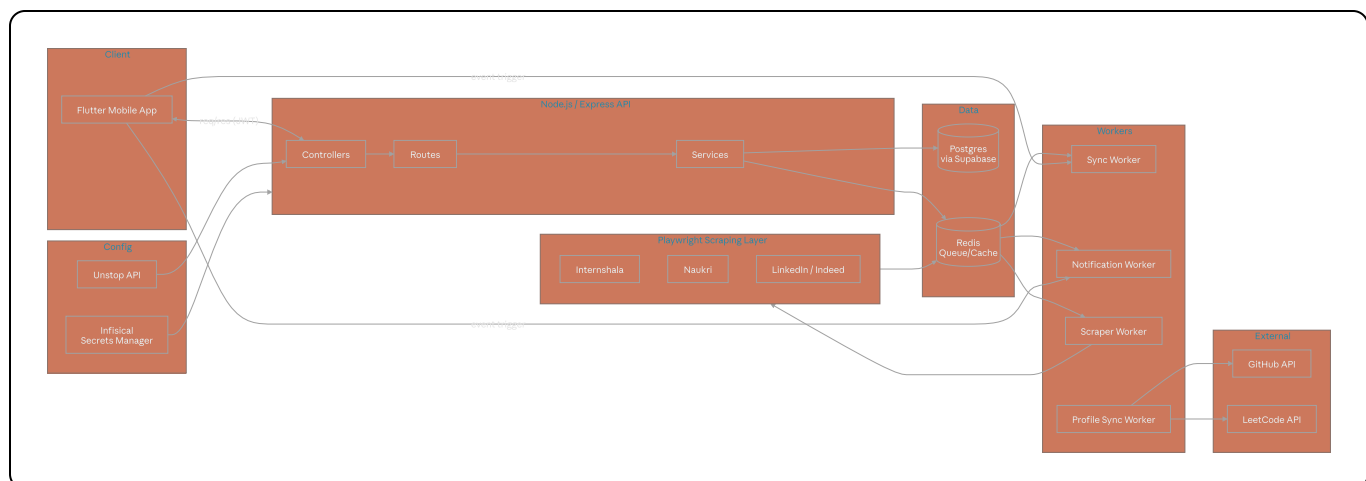
For FR9 (ATS Scanner):

- Given a resume file and a job description, when scanned, the system returns a numeric score (0-100).
- The system lists at least the keywords found in the JD but absent from the resume ("missing"), and those present.
- An "auto-tailor" action is offered when missing keywords exist.
- Given an unreadable/corrupt resume file, the system returns an explicit error rather than a fabricated score. (🔴 *Not available — actual validation logic does not exist yet; this is a recommended acceptance criterion.*)

3. System Architecture

🟡 **Inferred from the architecture diagram + tech stack.** The diagram shows a layered Node.js backend, a Postgres + Redis data layer, background workers, and Playwright-driven scrapers. No actual server code was reviewed, so component responsibilities below are reconstructed from box labels and arrows in the diagram, not verified against implementation.

3.1 High-Level Architecture



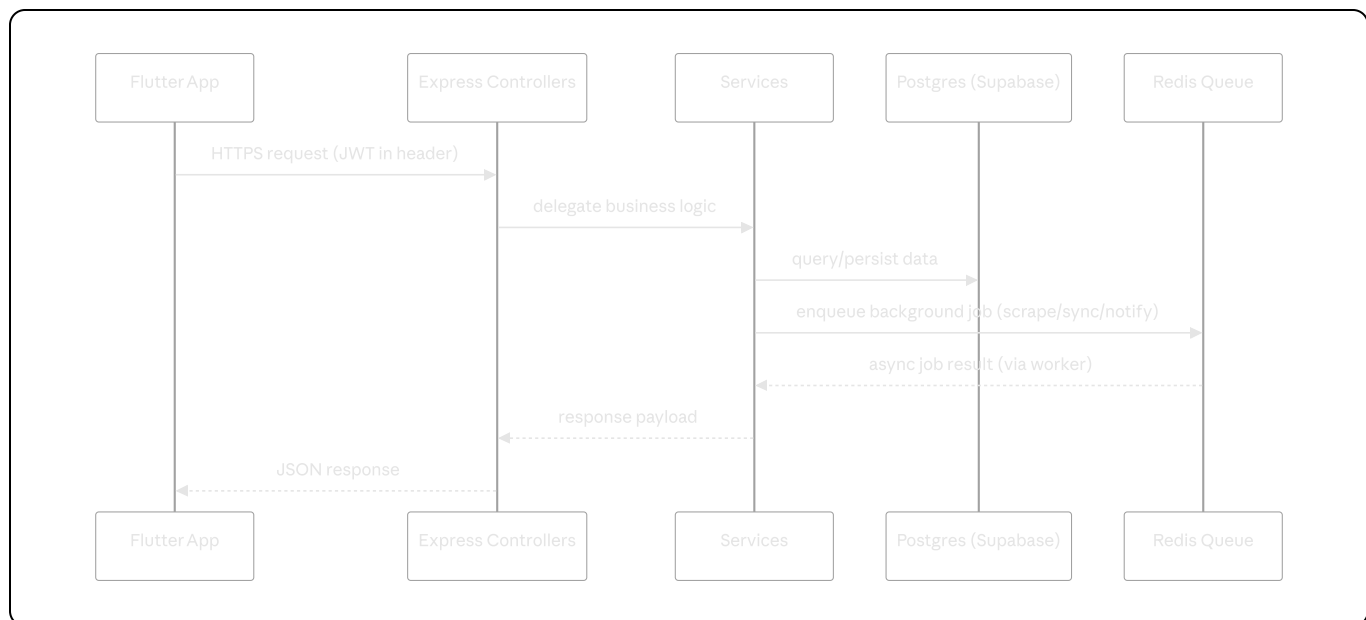
3.2 Layered Architecture ● Inferred

The diagram shows a classic 3-tier backend layering inside the API box:

1. **Controllers** — receive HTTP requests, orchestrate calls to Services, shape responses.
2. **Routes** — define URL-to-controller mappings (Express `Router`).
3. **Services** — business logic: scraping orchestration, scoring, matching.

This maps to a fairly standard MVC-without-views pattern common in Express APIs, though the actual file layout, naming, and boundaries are **not confirmed** — no route or controller files were provided.

3.3 Client-Server Flow ● Inferred



3.4 Request Lifecycle ● Inferred

1. Flutter app sends an authenticated HTTPS request.
2. Express receives it at a **Route**, which forwards to the matching **Controller**.
3. Controller validates input and calls the relevant **Service**.
4. Service reads/writes **Postgres** (via Supabase client) and/or pushes a job onto the **Redis** queue for asynchronous processing (e.g., scraping, notification dispatch).
5. Controller formats and returns the response.
6. Background **workers** (Sync, Notification, Scraper, Profile Sync) consume queued jobs independently of the request/response cycle, so scraping latency never blocks the mobile app.

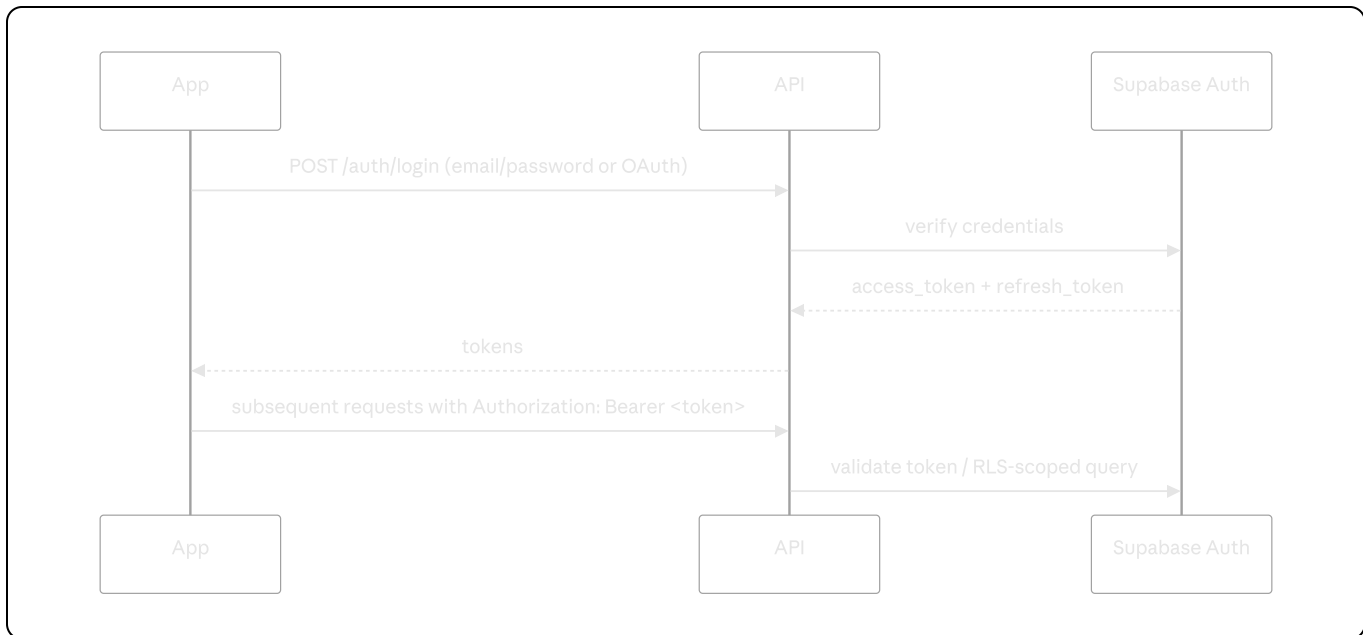
3.5 Authentication Flow ● Not available

The diagram shows a generic "req/res" arrow with a JWT-style icon between the phone and

the Controllers layer, implying **token-based authentication**, but:

- No auth middleware, token issuance endpoint, or Supabase Auth configuration was provided.
- Whether Supabase's built-in Auth (GoTrue) is used, or a custom JWT layer, **cannot be confirmed**.

Recommended flow to implement (not yet built):



3.6 Deployment Architecture ● Not available

No hosting configuration, Dockerfiles, or CI/CD pipeline files were supplied. Based on the stack (Supabase, Redis, Node workers), a typical deployment would separate:

- API service (stateless, horizontally scalable)
- Worker processes (scraper, sync, notification) — separately scaled since Playwright scraping is CPU/memory heavy
- Managed Postgres (Supabase-hosted)
- Managed Redis (e.g., Upstash/Redis Cloud) or self-hosted

This is a **recommendation**, not a description of an existing deployment.

4. Technology Stack

● **Derived** (from your description) where a technology is named; ● **Inferred** where a specific role is reconstructed from the diagram; version numbers are ● **not available** since no `package.json`/`pubspec.yaml` was supplied.

Layer	Technology	Version	Purpose	Why chosen (inferred)	Alternatives
Frontend	Flutter	● not available	Cross-platform mobile UI (iOS + Android from one codebase)	Single codebase, native-feel performance, rich widget ecosystem	React Native, native Swift/Kotlin
Frontend architecture	BLoC (Business Logic Component)	● not available	Separates UI from business logic/state	Testable, unidirectional data flow, Flutter-idiomatic	Riverpod, Provider, GetX
Backend runtime	Node.js	● not available	JavaScript runtime for the API server	Non-blocking I/O suits scraping + many concurrent light requests	Deno, Bun
Backend framework	Express.js	● not available	HTTP routing/middleware	Minimal, well-understood, huge ecosystem	Fastify, NestJS
Database / BaaS	Supabase (Postgres)	● not available	Relational data storage, likely also Auth/Storage	Managed Postgres + Auth + Realtime out of the box, generous free tier	Firebase, raw PostgreSQL + custom auth
Cache / Queue	Redis	● not available	Job queueing for scraper/sync/notification workers, caching	Fast in-memory store, mature queue ecosystem (BullMQ etc.)	RabbitMQ, AWS SQS
Browser automation	Playwright	● not available	Scrapes LinkedIn/Naukri/Internshala, automates registrations	Modern, multi-browser, resilient	Puppeteer, Selenium

Layer	Technology	Version	Purpose	Why chosen (inferred)	Alternatives
				selectors vs. Puppeteer	
Secrets management	Infisical	● not available	Centralized environment/secret management	Avoids hardcoded API keys/credentials in code	Doppler, AWS Secrets Manager, <code>.env</code> + vault
External job data	Unstop API	● not available	Official API source for opportunities (vs. scraping)	Reduces scraping surface/legal risk for at least one source	—
External developer data	GitHub API	● not available	Commit/repo/streak data for Career Health Score	Official, rate-limited but reliable	GraphQL API v4 variant
External developer data	LeetCode (unofficial API/scraping)	● not available	DSA practice data for Career Health Score	No official public API exists; typically GraphQL-scraped	—

5. Project Structure

● **Not available.** No repository, folder tree, or file listing was provided — only a single standalone HTML mockup file and an architecture sketch. A folder-by-folder breakdown of "purpose, responsibilities, dependencies, best practices" cannot be produced without an actual repo.

Recommended structure (to adopt once code exists, consistent with the architecture diagram's Controllers/Routes/Services split):

```

orbit/
├─ apps/
│   └─ mobile/                # Flutter app (BLoC architecture)
│       └─ lib/
│           └─ features/      # one folder per feature (feed, pipeline,
resume, coach...)
│           └─ core/          # shared widgets, theming, constants
│           └─ data/          # API clients, repositories
│           └─ pubspec.yaml
├─ services/
│   └─ api/                   # Node.js / Express backend
│       └─ src/
│           └─ controllers/
│           └─ routes/
│           └─ services/
│           └─ workers/       # scraper, sync, notification, profile-sync
│           └─ config/        # Infisical bootstrap, DB/Redis clients
│           └─ middleware/    # auth, error handling, validation
│           └─ package.json
├─ scrapers/                  # Playwright scripts per source
│   └─ linkedin/
│   └─ naukri/
│   └─ internshala/
└─ infra/                     # Docker/CI/deployment configs

```

6. Database Design

● **No schema, migration files, or Supabase table definitions were provided** — the Postgres/Supabase icon in the diagram confirms *that* a relational store is used, but not its structure. The entities below are **inferred purely from what the UI displays** (e.g., a job card needs a title, company, location, salary — so a jobs table almost certainly exists in some form), not from an actual schema.

6.1 Inferred Entity-Relationship Diagram

```

erDiagram
    USER ||--o{ APPLICATION : tracks
    USER ||--o{ SAVED_JOB : bookmarks
    USER ||--o{ RESUME : owns
    USER ||--|| CAREER_SCORE : has
    USER ||--o{ INTEGRATION : connects
    JOB ||--o{ APPLICATION : "applied via"

```

```
JOB ||--o{ SAVED_JOB : "saved as"
JOB }o--|| SOURCE : "sourced from"
RESUME ||--o{ APPLICATION : "used in"
APPLICATION ||--o{ INTERVIEW_ROUND : has
```

```
USER {
    uuid id PK
    string email
    string name
    string current_role
    string location
    jsonb skills
    jsonb job_preferences
}
JOB {
    uuid id PK
    string title
    string company
    string location
    string salary_range
    string job_type
    string source_id FK
    string external_url
    timestamp posted_at
    timestamp expires_at
    boolean is_expired
}
SOURCE {
    uuid id PK
    string name
    string type "api or scraped"
}
APPLICATION {
    uuid id PK
    uuid user_id FK
    uuid job_id FK
    uuid resume_id FK
    string stage "saved|applied|interview|offer|rejected"
    timestamp applied_at
    text notes
}
INTERVIEW_ROUND {
    uuid id PK
    uuid application_id FK
    string round_name
    timestamp scheduled_at
    string status
}
SAVED_JOB {
    uuid id PK
```

```

        uuid user_id FK
        uuid job_id FK
        timestamp saved_at
    }
    RESUME {
        uuid id PK
        uuid user_id FK
        string label
        string version
        string status "active|draft|archived"
        string file_url
    }
    CAREER_SCORE {
        uuid user_id PK FK
        int overall_score
        int github_score
        int leetcode_score
        int projects_score
        int applications_score
        int resume_score
        timestamp computed_at
    }
    INTEGRATION {
        uuid id PK
        uuid user_id FK
        string provider "github|leetcode|linkedin|indeed|naukri"
        boolean connected
        jsonb credentials_ref "pointer to Infisical secret, not raw creds"
    }
}

```

6.2 Entity Descriptions Inferred

Entity	Purpose
USER	The single application owner's profile, skills, and preferences
JOB	A normalized posting aggregated from Unstop API or a scraped source
SOURCE	Metadata about where a job came from (API vs. scraper, per-source config)
APPLICATION	Tracks a user's pipeline stage for a specific job, linked to the resume used
INTERVIEW_ROUND	Sub-records of an application (Technical screen, System design, HR, etc.)
SAVED_JOB	Wishlist/bookmark join table between USER and JOB
RESUME	A versioned resume file with performance stats

Entity	Purpose
CAREER_SCORE	The computed composite score shown on the Career Health dashboard
INTEGRATION	Connection status/reference for external accounts (GitHub, LeetCode, job boards)

6.3 Relationships, Keys, Constraints ● Inferred

- All primary keys assumed UUID (Supabase convention).
- `(APPLICATION.job_id → JOB.id)` and `(APPLICATION.resume_id → RESUME.id)` as foreign keys.
- `(stage)` on `(APPLICATION)` would need a `(CHECK)` constraint or enum type to match the four kanban columns observed in the UI (Saved, Applied, Interview, Offer) plus a terminal "Rejected"/"Expired" state.
- `(INTEGRATION.credentials_ref)` should **never store raw OAuth tokens/passwords directly in Postgres** — the presence of Infisical in the architecture suggests secrets are meant to live there, with Postgres holding only a reference or connection status. **This is a recommendation, not confirmed practice.**

6.4 Indexes ● Not available

No index definitions exist to review. Recommended: index `(JOB(source_id, posted_at))` for feed queries, `(APPLICATION(user_id, stage))` for kanban queries, and a full-text index on `(JOB(title, company))` for search.

7. Backend Documentation

● **No backend source files (controllers, services, repositories, DTOs, middleware) were provided.** The architecture diagram confirms a Controllers/Routes/Services layering exists conceptually, but none of the following can be documented from actual code:

- Controller implementations
- Service-layer business logic
- Repository/data-access patterns
- DTOs / request-response contracts
- Configuration files
- Utility modules
- Exception handling strategy
- Validation logic

What can be said (🟡 inferred): given Services sit between Controllers and both Postgres and Redis in the diagram, the intended pattern is that Services own all business logic (matching, scoring, scraping orchestration) while Controllers stay thin (parse request → call service → shape response) and Routes stay purely declarative. This is good separation-of-concerns practice **if implemented as drawn** — it should be verified against the real code once written.

No endpoint-level documentation (HTTP method, URL, auth requirement, request/response schema, status codes) can be produced without route files. See [Section 10](#) for a recommended API surface based on features instead of actual endpoints.

8. Authentication & Authorization

🔴 **Not available in detail.** The diagram implies a JWT-style request/response pattern between the phone and the Controllers layer, but no auth middleware, guard, token-refresh logic, or RBAC exists in what was provided, and there is only one user role (see 2.3), so role-based access control is likely **not applicable** to this project's current scope.

Recommended baseline (to implement):

- Use Supabase Auth (since Supabase is already the datastore) for email/password or OAuth (GitHub sign-in would double as the GitHub integration login).
 - Store only the Supabase session/access token client-side (Flutter secure storage), never long-lived credentials.
 - Validate the JWT on every API request in a single Express middleware before it reaches Controllers.
 - Use Supabase Row-Level Security (RLS) policies scoped to `user_id` so a compromised API bug can't leak cross-user data — relevant even for a single-user app if you ever add a second account.
 - Store all third-party credentials (GitHub/LeetCode tokens, scraping session cookies) in Infisical, referenced — never embedded — in Postgres rows.
-

9. Frontend Documentation

🟢 **Derived directly from** `orbit_full_ui.html`. This is the one layer of the project that can be documented with confidence, since the full UI mockup was provided.

9.1 Architecture 🟡 Inferred

The UI file is a static design reference (HTML/CSS "phone frame" mockups), not the actual

Flutter app — there is no Dart code, so real widget tree, routing table, or BLoC event/state classes cannot be extracted. Given the stated stack, the production app is expected to be a Flutter app using the **BLoC** pattern, with each screen below corresponding to one feature module and one Bloc/Cubit.

9.2 Design System ● Derived

Token	Value
Display font	Space Grotesk (headings)
Body font	Inter
Mono font	IBM Plex Mono (numbers/tags)
Navy	#142844
Amber (primary accent)	#FF8A3D
Paper (background)	#F6F4EF
Green (success)	#1A8A5A
Red (danger/expired)	#D94040
Purple (AI/coach accents)	#5B3FA6

9.3 Navigation Flow ● Derived

Bottom navigation is contextual per section rather than a single fixed 4-tab bar — the tab set changes between **Home/Search/Saved/Profile** (Discovery context) and **Home/Health/Coach/Profile** or **Home/Health/Saved/Profile** (Career Health context), and a separate **Home/Search/Pipeline/Profile** set on the Kanban screen. This suggests either a bottom-nav that adapts to the active feature area, or that not all four tabs are simultaneously reachable — this ambiguity should be resolved during actual navigation implementation.

9.4 Reusable Components (identified from CSS classes) ● Derived

Component	Used in
jobcard	Home feed, Saved jobs
kanban-col / kanban-item	Application pipeline

Component	Used in
<code>metric-card</code>	Pipeline, GitHub tracker, Profile
<code>score-ring</code> (SVG circular progress)	Career Health
<code>progress-track</code> / <code>progress-fill</code>	Career Health dimensions, GitHub languages, Skill gap
<code>skill-tag</code> (present/missing variants)	ATS Scanner, Skill Gap
<code>resume-card</code>	Resume Manager
<code>alert-strip</code> (info/warn/success variants)	Multiple screens — contextual nudges
<code>menurow</code> + <code>toggle</code>	Settings, Profile
<code>pill</code> (navy/amber/outline/green/red/purple variants)	Used app-wide for status tags

This is a well-factored, consistent component system — good practice to preserve when translating to actual Flutter widgets (each CSS class maps naturally to a reusable Flutter widget).

10. API Documentation

● **No route/controller code exists**, so a real API reference (headers, example requests/responses, validation, error codes) cannot be produced. Below is a **recommended API surface**, derived from what the UI needs to function — this is a specification to build against, not documentation of an existing API.

Method	Endpoint (proposed)	Purpose
POST	/auth/login	Authenticate user
GET	/jobs/feed	Ranked/matched job feed
GET	/jobs/:id	Job detail
POST	/jobs/:id/save	Bookmark a job
DELETE	/jobs/:id/save	Remove from saved
GET	/applications	List applications (kanban data)
POST	/applications	Create application (move job to "Applied")
PATCH	/applications/:id/stage	Move application between pipeline stages
POST	/applications/:id/rounds	Add/update an interview round
GET	/resumes	List resumes with performance stats
POST	/resumes	Upload a new resume version
POST	/resumes/:id/ats-scan	Run ATS scan against a job description
GET	/career-score	Fetch composite Career Health Score
GET	/skill-gap	Skill gap analysis vs saved/applied jobs
GET	/salary-intel?role=&city=	Salary benchmarking data
GET	/coach/today	Daily AI coach recommendations
POST	/integrations/:provider/connect	Connect GitHub/LeetCode/job boards
GET	/notifications	List notifications
PATCH	/settings/preferences	Update job preferences/notification toggles

Every one of these needs real request/response schemas, auth requirements, and error contracts defined once the backend is implemented — none of that can be fabricated responsibly here.

11. Core Features

●/● Derived from UI + functional dependency list

11.1 Aggregated Job Feed

- **Purpose:** Single ranked feed of opportunities from Unstop API + scraped sources.
- **Workflow:** Scraper/API workers fetch → dedupe/normalize → score against user skills → feed endpoint returns ranked list.
- **UI:** Home feed screen — match-percentage badge, source pills, quick filters (Remote, Full-time, skill, salary band).
- **Edge cases (recommended to handle):** duplicate postings across sources, source-site layout changes breaking a scraper, stale postings.

11.2 Application Pipeline (Kanban)

- **Purpose:** Track applications through Saved → Applied → Interview → Offer.
- **Workflow:** User moves a job card between columns; interview sub-stages (Technical screen, System design, HR) tracked with status and notes.
- **UI:** Kanban board + Application detail screen with recruiter mini-CRM (contact, last message).

11.3 Saved Jobs / Wishlist with Expiry Detection

- **Purpose:** Bookmark jobs for later; flag when a saved posting is no longer live.
- **Workflow:** Scheduled worker re-checks saved job URLs/IDs; marks `is_expired` and surfaces a warning pill + notification.

11.4 Career Health Score

- **Purpose:** Single 0-100 score composed of GitHub, LeetCode, Projects, Applications, and Resume sub-scores.
- **Workflow:** Profile Sync Worker periodically pulls GitHub/LeetCode activity; a scoring service (weights not specified in the UI — ● not available) recomputes the composite.
- **Edge case:** score should degrade gracefully if an integration is disconnected (UI shows LeetCode "Not connected" as a distinct state).

11.5 GitHub & LeetCode Tracking

- **Purpose:** Surface commit streaks, language mix, and contribution graph; nudge the user when a streak is at risk (UI: "No commits in 48 hours" alert).

11.6 Resume Manager & ATS Scanner

- **Purpose:** Maintain multiple resume versions, track interview-conversion rate per version, and score a resume against a specific job description with keyword gap detection and one-click "auto-tailor."

11.7 Skill Gap Analyzer

- **Purpose:** Compare declared skills against demand across the user's saved/applied job set (UI states "Analysed 500 Flutter jobs"), surfacing high-priority missing skills.

11.8 Salary Intelligence

- **Purpose:** Median and top-25% compensation by role, broken down by city, plus a 5-year YoY trend chart.

11.9 AI Career Coach

- **Purpose:** Daily, short, prioritized recommendations (e.g., "solve 2 DP problems," "push a commit," "apply to 6 more jobs") based on the user's current activity vs. an implied target (12 applications/week in the mockup).

11.10 Learning Roadmap & Notes/PDF Export

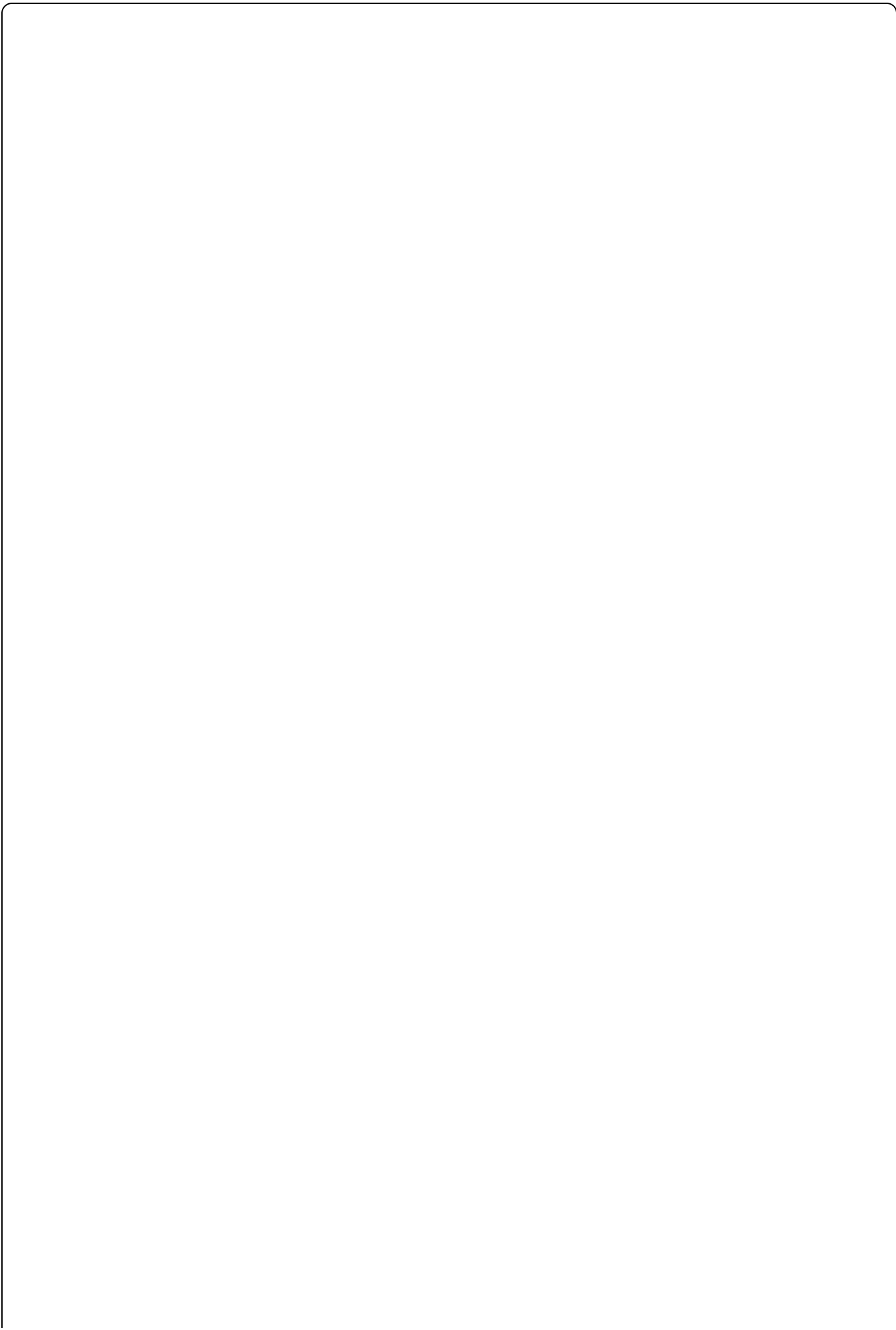
- **Purpose:** Per the functional dependency list — generate a structured roadmap and exportable notes/PDF.  **Screen exists in the nav but detailed content was truncated/not fully inspected in the UI file; behavior beyond the section heading is not confirmed.**

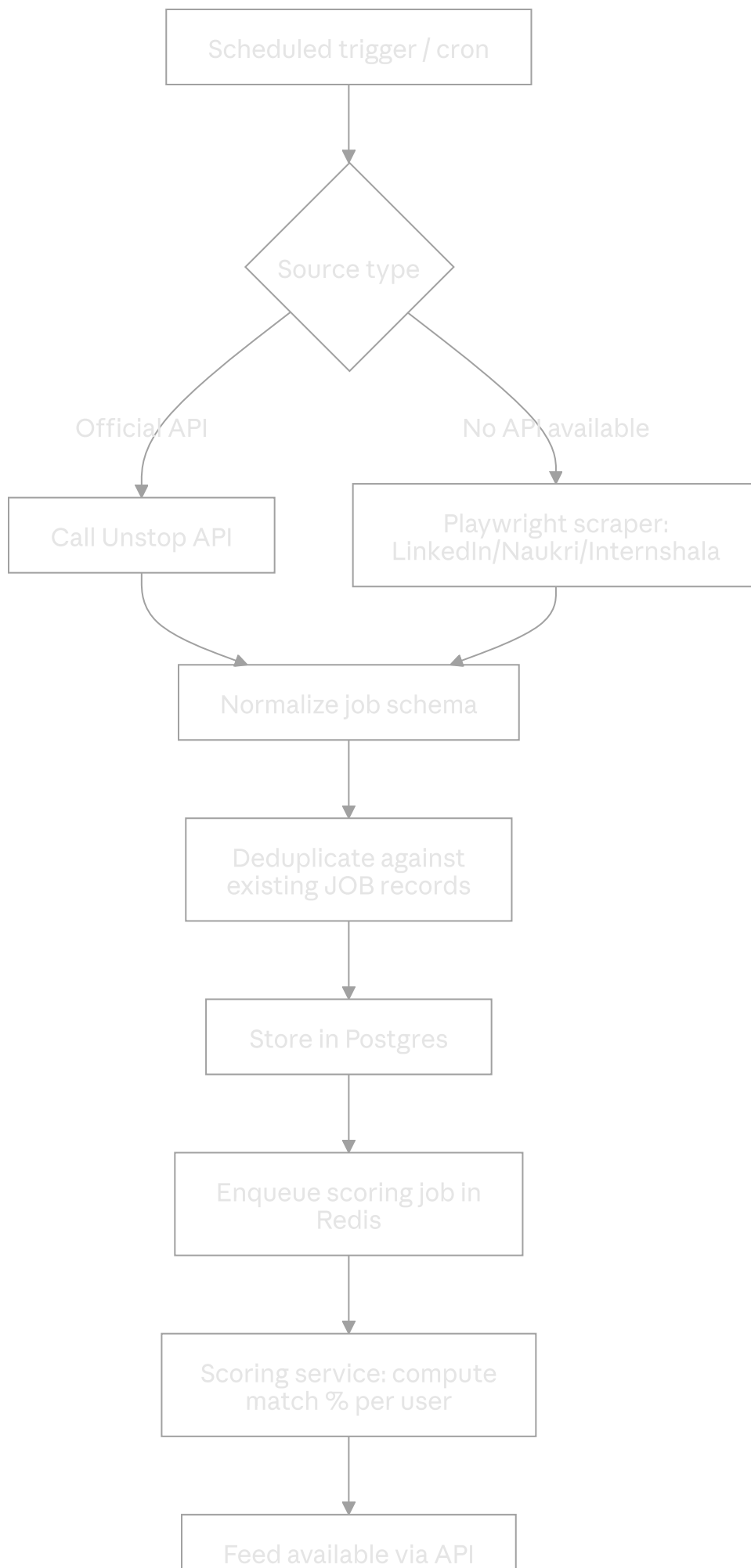
11.11 Automated Job Registration

- **Purpose:** Use Playwright to script application submission on external portals.
 -  **Important caveat (not a technical limitation, a compliance one):** most job boards' Terms of Service explicitly prohibit automated account actions/scraping. This should be reviewed against each target site's ToS before implementation, particularly for LinkedIn, which actively detects and bans automation.
-

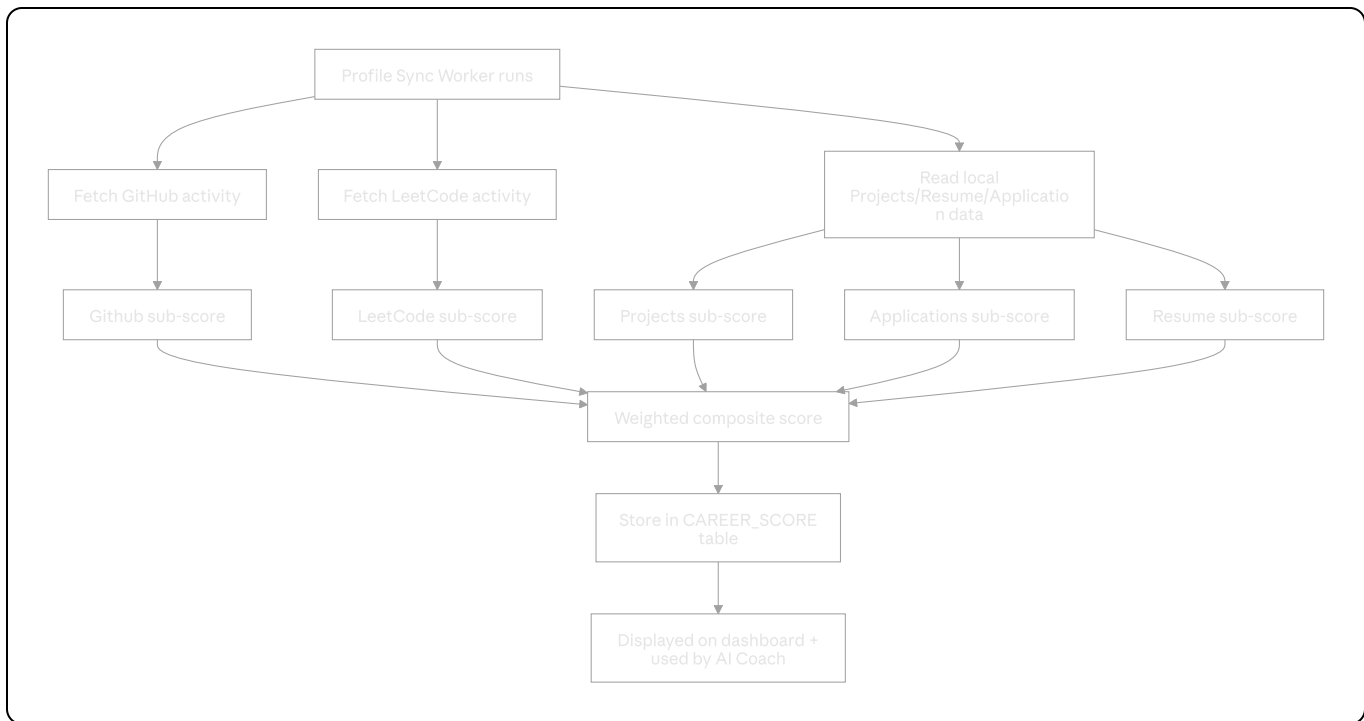
12. Algorithms & Workflows

12.1 Job Aggregation Flow Inferred



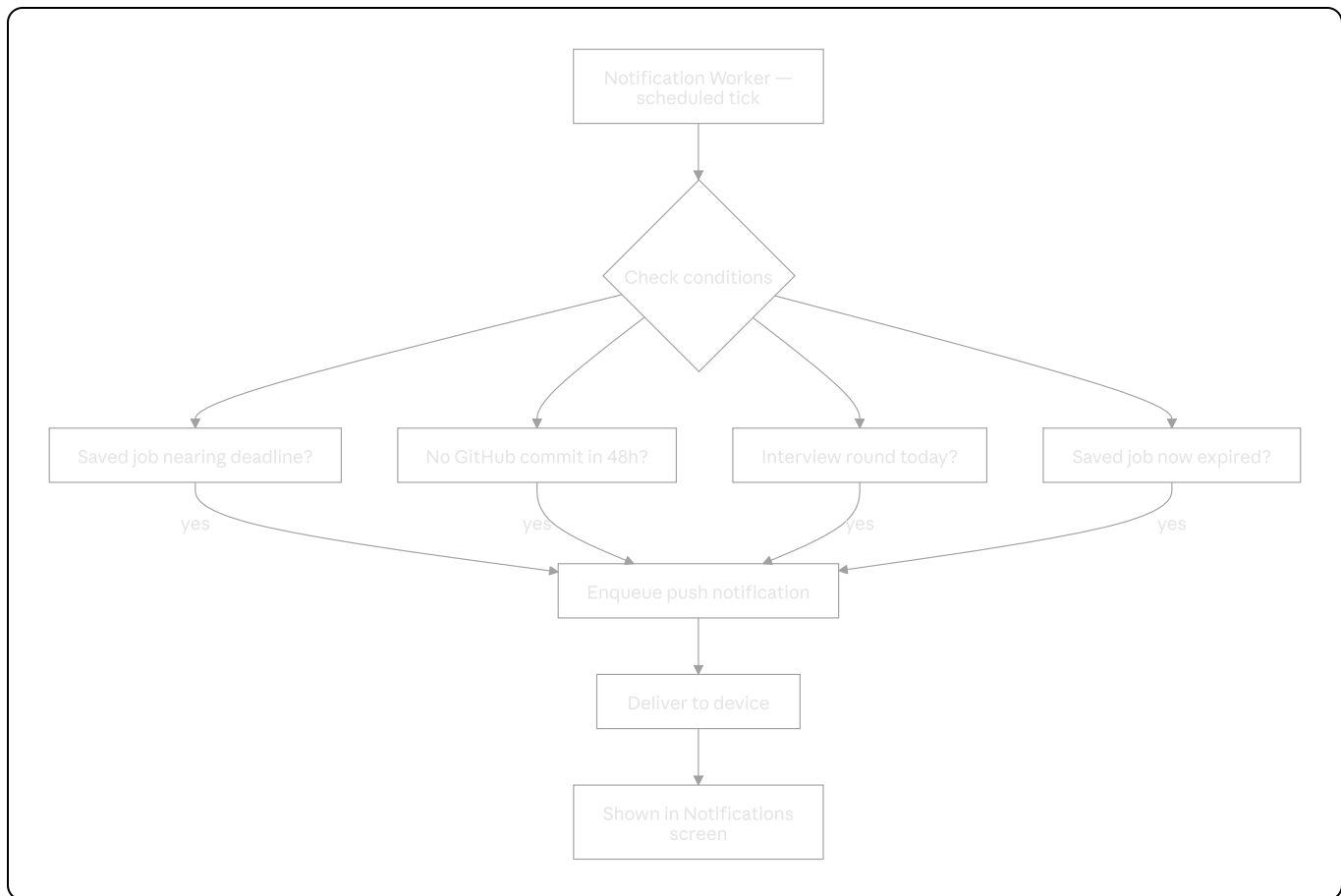


12.2 Career Health Score Computation ● Inferred



(Exact weighting formula is not specified anywhere in the provided materials — ● not available.)

12.3 Deadline / Streak Notification Flow Inferred



12.5 Custom Algorithms 🔴 Not available

No scoring formulas, matching weight functions, or ranking algorithms were provided in code form. The percentages shown in the UI (e.g., "91% match," "Career Score 78") are illustrative mockup data, not the output of a real algorithm.

13. UI/UX Documentation

🟢 Fully derived from `orbit_full_ui.html`. 24 screens across 8 sections.

13.1 Section 01 — Onboarding

Screen	Purpose	Components	Interaction
Splash	Brand moment	Logo mark, CTA to sign up/sign in	Tap "Get started"
Onboarding — Goals	Personalize feed from day one	3 selectable goal cards (Land first job / Switch roles / Level up salary), progress bar (Step 1 of 3)	Single-select, tap Continue
Onboarding — Skills	Seed the skill-gap engine	Tag-style multi-select chips + "Add skill", info strip confirming count	Multi-select, tap Continue

13.2 Section 02 — Job Discovery

Screen	Purpose	Components	Interaction
Home feed	Primary discovery surface	Greeting header, "342 new jobs · 6 sources" sync pill, search bar, quick-filter chips, ranked job cards with match-% badge and source attribution	Tap card → Job detail; tap bookmark to save
Job detail	Full posting view	(not fully inspected — truncated in review)	—
Search & filters	Refine discovery	Filter controls	—

13.3 Section 03 — Application Pipeline

Screen	Purpose	Components	Interaction
Application kanban	Visual pipeline	Metric cards (Applied/Active/Offers), 3+ columns (Saved, Applied, Interview[, Offer]) with card counts	Drag-and-drop implied by kanban paradigm (not explicit in static mockup)
Application detail	Per-application deep dive	Interview rounds timeline (dot + status pill per round), notes block, recruiter mini-CRM row with "Message" action	Back navigation, tap recruiter row
Saved jobs	Wishlist management	Tabs (All/Applied/Expired), job cards with delete/trash action, status pills including "Expired" warning	Tab switch, delete from list

13.4 Section 04 — Career Health Dashboard

Screen	Purpose	Components	Interaction
Career health	Composite readiness score	Circular score ring (78/100), weekly delta, 5 dimension progress bars (GitHub, LeetCode, Projects, Applications, Resume)	Tap info icon for methodology
GitHub tracker	Coding activity detail	Metric cards (Commits/Repos/Stars), streak alert strip, 7-day streak dot row, language breakdown bars, contribution heatmap grid	Back navigation
LeetCode tracker	DSA practice detail	(section present, detail not fully inspected)	—

13.5 Section 05 — Resume Tools

Screen	Purpose	Components	Interaction
Resume manager	Compare resume versions	Resume cards with version, status pill (Active/Draft), per-resume stats (Applied/Interviews/Rate), best-performer success banner	Tap "+Add" to upload new version

Screen	Purpose	Components	Interaction
ATS scanner	Score resume vs. JD	Large score display (81), missing/present keyword tag lists, auto-tailor CTA and info strip	Tap "Auto-tailor resume"
Skill gap analyzer	Skills vs. market demand	Per-skill demand bars with present/missing icon, red bars for high-priority gaps, warning strip	Tap "View learning roadmap"

13.6 Section 06 — Salary Intelligence

Screen	Purpose	Components	Interaction
Salary intelligence	Compensation benchmarking	Median/Top-25% summary card, per-city salary band bars with percentile marker, 5-year YoY mini bar chart	Back navigation
AI coach	Daily guidance	Coach message card (weekly application count vs. high-performer benchmark, weakest skill callout), 3 recommended-action cards	Tap action card (implied deep link)
Learning roadmap	(section present; detailed content not fully inspected)	—	—

13.7 Section 07 — Profile & Settings

Screen	Purpose	Components	Interaction
Profile	Identity + integrations hub	Avatar/name/role, metric cards (Applied/Saved/Score), job-source toggles (LinkedIn/Indeed/Naukri), integration rows (GitHub "Live", LeetCode "Connect")	Toggle sources, tap "Connect"
Notifications	Notification center	Cards per event type (interview scheduled, streak at risk, jobs expired, new matches) with unread dot and relative timestamp	"Mark all read"
Settings	Preferences hub	Grouped menu rows: Job preferences (locations/salary/type), Notification toggles,	Tap row to drill in, toggle

Screen	Purpose	Components	Interaction
		Account (resumes, sign out)	switches

13.8 Section 08 — Empty States

Screen	Purpose
Empty — Pipeline	Shown when no applications exist yet
Empty — Saved	Shown when no jobs are bookmarked
Empty — No results	Shown when search/filter returns nothing

(Empty-state screens confirm the design accounted for zero-data conditions — good UX practice — though their exact copy/illustration was not deeply inspected here.)

13.9 Business Rules Reflected in the UI ● Derived

- A job can be simultaneously "Saved" and later "Applied" (state transition, not a duplicate record) — evidenced by the Saved Jobs tab filter "Applied".
- Expired jobs are visually distinguished (red "Expired" pill) rather than silently removed, preserving user history.
- Resumes without any applications yet show "0%" rate rather than "N/A," implying the UI computes rate as $\frac{\text{interviews}}{\text{applied}}$ and defaults gracefully when $\frac{\text{applied}}{\text{applied}} = \frac{0}{0}$ — **this specific default behavior should be explicitly implemented, not assumed, once the calculation is coded.**

14. Security Analysis

● **Inferred from the presence of Infsical in the stack; no actual security code (input sanitization, headers, rate limiting) was provided to audit.**

Concern	Status
Secrets management	● Addressed at the architecture level via Infisical — keep it that way; never commit <code>.env</code> files with real keys
Authentication	● Not implemented/documented — see Section 8 recommendations
SQL injection	● Likely low-risk if using Supabase's client library (parameterized queries by default), but only true if raw SQL string concatenation is avoided in custom queries — must be verified in actual code
XSS	● N/A to review — no rendering code (Flutter renders natively, not via HTML injection, which reduces classic XSS risk but doesn't eliminate it if webviews are used)
CSRF	● Lower risk for a token-based mobile API (no cookie-based session), but confirm once auth is implemented
Scraper credential storage	⚠ If Playwright automation logs into LinkedIn/Naukri/Internshala with the user's own credentials, those credentials must live in Infisical, never in Postgres or app code, and session cookies should be encrypted at rest
Rate limiting	● Not implemented/documented — recommended on all public-facing endpoints, and especially on scraper trigger endpoints to avoid tripping source-site anti-bot defenses
Automated application submission	⚠ This is a security and legal consideration: storing and replaying login sessions for third-party sites on the user's behalf increases the blast radius if Orbit's own infrastructure is compromised

15. Performance Optimization

● **Recommendations**, since no actual query plans, caching code, or load-test results exist:

- **Caching:** Cache the ranked job feed per user for a short TTL (e.g., 5-10 minutes) in Redis rather than recomputing match scores on every request.
- **Database:** Add the indexes proposed in 6.4; use pagination (cursor-based) on `/jobs/feed` and `/applications` rather than returning full tables.
- **Scraper isolation:** Playwright workers are memory/CPU heavy — run them in a separate process/container pool from the API so scraping spikes never starve API

response times (already implied as intended by the worker/queue split in the diagram).

- **Connection pooling:** Use Supabase's pooled connection string (PgBouncer) for the API layer, especially under serverless/many-instance deployment.
 - **Image/asset optimization:** Company logos and resume thumbnails should be served from Supabase Storage with a CDN, not proxied through the API.
-

16. Error Handling

● **Not available** — no exception-handling middleware, error classes, or logging configuration exist in what was provided.

Recommended baseline:

- A single Express error-handling middleware that converts thrown errors into a consistent `{ error: { code, message } }` JSON shape.
 - Distinct handling for: validation errors (400), auth failures (401/403), not-found (404), scraper/upstream failures (502/503 — since a source site being down shouldn't surface as a generic 500), and unexpected errors (500, logged with stack trace but not exposed to the client).
 - Scraper failures specifically should be caught per-source, so one broken scraper (e.g., LinkedIn changes its DOM) doesn't halt aggregation from the other sources.
-

17. Deployment Guide

● **Not available** — no Dockerfiles, environment variable lists, or CI/CD configuration were supplied.

Prerequisites (recommended, based on stack):

- Node.js LTS runtime
- Flutter SDK (matching the app's target Dart SDK, once `pubspec.yaml` exists)
- Supabase project (Postgres + Auth + Storage)
- Redis instance
- Infisical project with environment secrets configured
- Playwright browser binaries installed in the worker environment (`(npx playwright install)`)

Environment variables to plan for: Supabase URL/keys, Redis connection string, Unstop API key, GitHub OAuth client ID/secret, Infisical project token — none of these exist yet as an actual `.env.example`, so this list is a starting point, not a confirmed requirement set.

Actual local-run instructions, build process, and cloud deployment steps cannot be written until the codebase exists.

18. Testing

● **Not available** — no test files or testing framework configuration were provided.

Recommended strategy:

- **Unit tests** for scoring/matching logic (pure functions — highest value, easiest to test).
- **Integration tests** for each Service against a test Postgres instance (or Supabase local dev stack).
- **Scraper tests** using recorded fixtures (HTML snapshots) rather than hitting live sites in CI, since LinkedIn/Naukri layouts change and live scraping in CI is fragile and ToS-risky.
- **Widget tests** in Flutter for each screen (BLoC's testability is a strong argument for the chosen architecture).
- **Manual testing** checklist should explicitly cover the empty states (Section 13.8), since those are easy to forget once real data exists.

Known limitations of current state: there is nothing to test yet — this section should be revisited once the API and app are implemented.

19. Software Engineering Practices

● **Inferred from the architecture diagram's intent, not verified in code:**

- **Separation of Concerns:** Controllers/Routes/Services split suggests an intended layered architecture.
- **Single Responsibility (per worker):** Sync, Notification, Scraper, and Profile Sync are separate workers rather than one monolithic background job — good practice if maintained.
- **BLoC pattern (frontend):** Encourages unidirectional data flow and testable business logic, separate from widgets.
- **Config externalization:** Infisical use suggests adherence to the 12-factor "config in environment, not code" principle.

Cannot be assessed without code: actual SOLID adherence, design patterns used (Repository, Factory, Strategy, etc.), coding standards/linting, dependency injection usage, or code duplication — these require a real codebase to review.

20. Future Enhancements

- **Scalability:** Move scraper workers to a horizontally scalable job-queue system (e.g., BullMQ on Redis, already partially implied) with per-source rate limiting.
 - **Security:** Implement RLS on all Supabase tables; rotate scraping session credentials automatically.
 - **Feature roadmap:** Complete the Learning Roadmap/PDF export feature; add in-app messaging with recruiters (currently only a "Message" deep-link stub in the UI); support multiple job-seeker profiles per install (e.g., separate "Backend" vs "Flutter" personas).
 - **Cloud migration/microservices:** If usage grows beyond personal use, split the Scraper Worker into its own deployable service with independent scaling, since it has the heaviest and most variable resource profile.
 - **AI integration:** The "AI Coach" is currently rule-based per the UI copy ("high performers average 12/week"); a genuine LLM-driven coach could generate more personalized, context-aware guidance.
 - **Monitoring/analytics:** Add structured logging and a dashboard (e.g., scraper success/failure rate per source) — critical given how fragile scraping-based data pipelines are.
-

21. Challenges Faced

● **Anticipated, based on the architecture — these are foreseeable challenges for this class of system, not challenges reported by you:**

Challenge	Why it's likely	Possible mitigation
Scraper fragility	LinkedIn/Naukri/Internshala frequently change DOM structure and actively detect automation	Use resilient selectors, fallback to official APIs where available (as done with Unstop), monitor scraper health
ToS / legal risk of automated registration	Most job boards prohibit automated account actions	Restrict automation to sources with explicit API support; treat scraped-source "auto-apply" as high-risk/manual-review
Rate limiting from external APIs	GitHub and LeetCode both throttle unauthenticated/high-frequency requests	Cache aggressively, use authenticated requests, respect backoff headers
Scoring subjectivity	"Career Health Score" and "match %" require a defensible weighting formula, not arbitrary numbers	Document and version the scoring formula explicitly; allow it to evolve without silently changing historical scores
Worker/queue complexity	Four distinct workers (Sync, Notification, Scraper, Profile Sync) add operational overhead for a personal project	Consider consolidating into fewer worker types initially, splitting only when load justifies it

22. Conclusion

22.1 Project Summary

Orbit is designed as a personal, aggregation-first job search system that unifies multiple job boards into a single ranked feed and layers on career-readiness tooling (resume ATS scanning, skill-gap analysis, GitHub/LeetCode tracking, and a composite Career Health Score) that no single job board offers today.

22.2 Current State

The project currently exists as: (1) a complete, well-designed 24-screen Flutter UI mockup with a consistent design system, and (2) a sketched system architecture showing a layered Node.js/Express backend backed by Postgres, Redis, and Playwright-based scraping workers. **No backend, database, or Flutter application code has been written yet.**

22.3 Recommended Next Steps

1. Stand up the Supabase project and implement the schema proposed in Section 6, refining it as real requirements surface.
2. Build the Express API's Controllers/Routes/Services skeleton per Section 3.2, starting with `/jobs/feed` and `/applications`.
3. Implement one scraper (start with the source with the most permissive ToS) plus the Unstop API integration before attempting all four sources.
4. Convert the HTML mockup into actual Flutter widgets/BLoCs screen-by-screen, using Section 13 as the screen inventory.
5. Revisit this document once real code exists — every / section above should become  as the corresponding component is built, and this documentation should be regenerated against the actual repository at that point for genuine accuracy.

22.4 Learning & Technical Impact

Building Orbit end-to-end would demonstrate full-stack competency (Flutter/BLoC, Node/Express, Postgres, Redis-backed background processing, browser automation) plus product-thinking skills (translating a personal pain point into a scoped feature set) — strong material for a final-year project or portfolio piece, provided the documentation is updated to reflect the real implementation rather than the current design-stage snapshot.